Java Design Patterns - Interview Notes

Prepared for Lakshmi Narayana K | E-commerce examples | October 2026

Simple explanations of what each pattern means, why it is useful, where it fits, Java examples, and interview answers. Includes the questions and clarifications from our design-pattern learning sequence.

Learning sequence

1. Singleton
2. Repository
3. Proxy
4. Strategy
5. Simple Factory
6. Builder
7. Adapter
8. Decorator
9. Publish-Subscribe
10. Saga Choreography

How to use these notes

Each pattern starts with what it is, the earlier problem, why it was introduced, advantages, exact usage and limitations. Then read the Java example and interview answer. Examples demonstrate possible implementations; they do not prove that a feature exists in your current project. Say "I use" only when your code supports that claim.

Singleton, Proxy, Strategy, Builder, Adapter and Decorator are classic object design patterns. Repository is a persistence pattern. Publish-Subscribe and Saga are messaging/distributed-system patterns. The payment factory shown here is Simple Factory, not the formal GoF Factory Method pattern.

Java snippets are focused examples. Standard imports, entities and framework configuration are omitted where they are not the topic. Use BigDecimal or validated minor-unit amounts for real money.

1. Singleton - What, Why and Advantages

1. What is it?

One shared instance with a controlled access point.

2. What was the problem before?

Each new AppConfig() creates a separate object. If configuration is intended to be shared, separate copies can lead to inconsistent state or unnecessary creation.

3. Why did we introduce it?

Provide the same instance wherever that shared object is required. Use this only when shared lifetime is appropriate.

4. What are the advantages?

Reuses one object; provides consistent access; centralizes its lifetime. It is not a substitute for connection pooling or thread safety.

5. Where exactly does it fit?

Spring normally shares one ProductService bean per definition per application context. Keep customer-specific data out of instance fields.

6. When should we be careful?

Avoid unnecessary global state. Shared mutable fields need concurrency control.

Continue to the next page for the Java example, detailed usage and interview questions.

1. Singleton - Example and Interview Questions

What and why

Singleton provides one instance of a class and access to it. It is useful when one shared instance is appropriate. It does not automatically make shared mutable data thread-safe.

```
class AppConfig {
    private static final AppConfig INSTANCE = new AppConfig();
    private AppConfig() { }
    public static AppConfig getInstance() {
        return INSTANCE;
    }
}
// Inside main:
AppConfig a = AppConfig.getInstance();
AppConfig b = AppConfig.getInstance();
System.out.println(a == b); // true
```

Private constructor prevents ordinary external construction. Static final field holds the shared instance. getInstance() returns it. This eager version is initialized safely during class initialization; reflective or serialization concerns need additional design where applicable.

Exact project usage

Spring services such as ProductService are singleton-scoped by default: one instance per bean definition per application context. Spring manages this scope; you do not implement getInstance() or a private constructor in a normal service.

Q: Is there only one instance across all servers?
No. Three running Product Service applications normally have three contexts and their own bean instances.

Q: Is Singleton thread-safe?
Instance creation can be safe while its mutable fields remain unsafe. Avoid storing currentCustomerId or request-specific state in a shared service. Use parameters and local variables.

Interview answer

"Singleton provides one shared instance. In Spring, beans are singleton-scoped by default per bean definition per application context. I keep services stateless to avoid sharing request data between concurrent calls."

2. Repository - What, Why and Advantages

1. What is it?

A boundary for loading and saving domain data.

2. What was the problem before?

Mixing SQL or persistence details into business methods makes order rules harder to read and database code easy to duplicate.

3. Why did we introduce it?

Move persistence access behind a focused repository so services can concentrate on business rules.

4. What are the advantages?

Clear separation; reusable queries; business logic is easier to unit-test with a mocked repository.

5. Where exactly does it fit?

ProductService calls ProductRepository.findById(id); JPA/Hibernate performs PostgreSQL access.

6. When should we be careful?

Queries still need performance tuning, and changing databases can still require repository changes.

Continue to the next page for the Java example, detailed usage and interview questions.

2. Repository - Example and Interview Questions

What and why

Repository separates persistence access from business logic. ProductService decides business rules; ProductRepository provides operations to load and save product data.

```
public interface ProductRepository
        extends JpaRepository&lt;Product, Long&gt; { }

@Service
public class ProductService {
    private final ProductRepository repository;
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }
    public Product getProduct(Long id) {
        return repository.findById(id).orElseThrow(() -&gt;
            new ProductNotFoundException(id));
    }
}
```

Product is the entity and Long is its ID type. ProductNotFoundException is an application-defined exception. Spring Data supplies repository implementations, including standard save(), findById() and deleteById() operations.

Exact project usage

ProductRepository accesses products and stock; OrderRepository saves orders and status changes; UserRepository finds users by email. A PaymentRepository stores payment records if implemented. The usual path is Controller -> Service -> Repository -> Database.

Q: Is Repository the same as Hibernate?
No. Repository is a design pattern. Hibernate is an ORM implementation, commonly used underneath Spring Data JPA to access PostgreSQL.

Q: Why not put database code in the service?
Separating persistence avoids mixing database details with business rules and makes focused testing and maintenance easier. A repository still does not make every database change transparent.

Interview answer

"I use Repository through Spring Data JPA. ProductRepository handles persistence while ProductService contains business logic. This keeps database access separate and easier to test."

3. Proxy - What, Why and Advantages

1. What is it?

An intermediary that delegates to a real object and adds access or surrounding behavior.

2. What was the problem before?

Manually writing transaction start, commit and rollback around every business operation repeats infrastructure code and risks inconsistent handling.

3. Why did we introduce it?

Apply transaction behavior around service methods through a proxy instead of repeating it in each method.

4. What are the advantages?

Separates infrastructure from business logic; applies consistent behavior; reduces repeated transaction code.

5. Where exactly does it fit?

Controller calls the injected OrderService proxy; it manages the transaction around createOrder().

6. When should we be careful?

Default Spring proxy advice is bypassed by direct self-invocation. A local transaction does not undo another service commit.

Continue to the next page for the Java example, detailed usage and interview questions.

3. Proxy - Example and Interview Questions

What and why

A proxy is an intermediary that represents an actual object. It can add behavior before or after delegating to it. Think of a security guard checking access before letting someone meet an employee.

```
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    // Save related order items in the same transaction.
}
```

When another bean calls this method through the Spring proxy, transaction infrastructure starts or joins a transaction, invokes the business method, and then commits or rolls back as appropriate.

```
// Simplified explanation, not Spring implementation:
startTransaction();
try {
    actualService.createOrder(order);
    commitTransaction();
} catch (RuntimeException ex) {
    rollbackTransaction();
    throw ex;
}
```

Exact project usage

The injected OrderService is typically a proxy when transaction advice applies. OpenFeign also creates a proxy for ProductClient: calling getProduct(id) causes an HTTP request rather than running a handwritten client implementation.

Q: Who starts and commits the transaction?
Spring transaction infrastructure invoked through the proxy. Your method contains business and persistence operations.

Q: Does an internal call apply @Transactional?
With default proxy-based handling, this.createOrder(...) bypasses the proxy. The called annotation is not applied; an existing transaction may still be active.

Q: Which failures roll back by default?
RuntimeException and Error. Checked exceptions need suitable rollback configuration when rollback is required.

Q: Does this roll back other services or Kafka?
No. A local database transaction does not automatically reverse remote service commits or independently published Kafka events.

Interview answer

"Spring uses proxies to apply transaction management around my business methods. Feign supplies client proxies that translate interface calls into HTTP requests."

4. Strategy - What, Why and Advantages

1. What is it?

Different interchangeable implementations of the same operation.

2. What was the problem before?

A single payment method containing all card, UPI and wallet processing becomes large. Changes to one provider can disturb unrelated logic.

3. Why did we introduce it?

Give each payment behavior its own class implementing Payment; let checkout delegate to the supplied implementation.

4. What are the advantages?

Isolated behavior; focused tests; new processors can be added without embedding their logic in checkout.

5. Where exactly does it fit?

CardPayment and UpiPayment implement pay(amount). Payment Service delegates to the selected object.

6. When should we be careful?

A caller or selector still chooses the object. Strategy does not mean pay(amount) independently chooses a payment type.

Continue to the next page for the Java example, detailed usage and interview questions.

4. Strategy - Example and Interview Questions

What and why

Strategy packages different ways of doing the same task behind a common interface. A checkout can use card payment or UPI payment without containing both implementations.

```
interface Payment {
    void pay(int amount);
}
class CardPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Card: " + amount);
    }
}
class UpiPayment implements Payment {
    public void pay(int amount) {
        System.out.println("UPI: " + amount);
    }
}
class CheckoutService {
    private final Payment payment;
    CheckoutService(Payment payment) {
        this.payment = payment;
    }
    void checkout(int amount) {
        payment.pay(amount);
    }
}
// Inside main:
new CheckoutService(new UpiPayment()).checkout(1000);
```

Exact project usage

Inside Payment Service, different payment processors could implement Payment. The workflow delegates to the selected processor and then publishes the payment result. This is a possible extension if your current service has only one processor.

Q: What does interchangeable mean?
Checkout can receive CardPayment or UpiPayment and call the same pay() operation, provided both honor the common contract.

Q: Does pay(1000) choose the payment type?
No. The actual object was selected earlier. Payment p = new UpiPayment(); p.pay(1000) runs UpiPayment.pay(). The argument 1000 is only the amount.

Q: Strategy versus polymorphism?
Runtime polymorphism dispatches the call to the actual implementation. Strategy is a design that uses interchangeable behavior, often through polymorphism.

Interview answer

"Strategy lets us use different ways to perform the same operation. Card and UPI processors can implement one payment interface, and checkout delegates to the supplied implementation."

5. Simple Factory - What, Why and Advantages

1. What is it?

One place that creates or provides the appropriate object from an input.

2. What was the problem before?

Checkout and subscription renewal can repeat the same CARD/UPI conditions. Adding Wallet then requires changes in many places.

3. Why did we introduce it?

Move those creation rules into PaymentFactory and let callers request a Payment.

4. What are the advantages?

Less duplicate creation logic; callers do not know concrete constructors; creation changes are localized.

5. Where exactly does it fit?

getPayment("UPI") returns UpiPayment; then payment.pay(1000) runs the selected behavior.

6. When should we be careful?

The simple factory still changes for a new type. With only one trivial creation point, it may add needless complexity.

Continue to the next page for the Java example, detailed usage and interview questions.

5. Simple Factory - Example and Interview Questions

What and why

A Simple Factory centralizes creation of appropriate objects based on input. Without it, checkout and subscription renewal may each repeat the same CARD/UPI conditions and new expressions.

```
class PaymentFactory {
    static Payment getPayment(String type) {
        if ("CARD".equalsIgnoreCase(type)) {
            return new CardPayment();
        }
        if ("UPI".equalsIgnoreCase(type)) {
            return new UpiPayment();
        }
        throw new IllegalArgumentException(
            "Unsupported payment type: " + type);
    }
}
// Inside calling code:
Payment payment = PaymentFactory.getPayment("UPI");
payment.pay(1000);
```

Exact project usage

Payment Service can use a factory/provider to obtain the processor matching a payment type. In Spring, processors with injected dependencies are usually managed beans; select those beans rather than creating them with new and bypassing Spring wiring.

Q: What was the disadvantage before Factory?
Selection code was duplicated in many callers. Adding Wallet required updating all of them; a missed caller would not support it.

Q: What is the advantage?
Creation rules live in one place. Callers use Payment and do not need to know each concrete constructor.

Q: Did the if/else disappear?
No. In this example it moved into one place. Adding a type still changes the factory, but not every caller.

Q: Is Factory always necessary?
No. One simple creation point may not justify an additional class.

Q: Is this GoF Factory Method?
No. This is Simple Factory. Factory Method defines a creation method whose implementation can be overridden by subclasses.

Interview answer

"A factory centralizes object creation. PaymentFactory can return CardPayment or UpiPayment based on input, avoiding repeated creation logic in callers."

Factory versus Strategy - Your Follow-ups

Q: What exactly is the difference?
Factory is about obtaining or creating an object. Strategy is about organizing interchangeable ways of performing an operation.

```
Payment payment = PaymentFactory.getPayment("UPI");
// Factory returns a UpiPayment object.

payment.pay(1000);
// Java calls that object's UPI-specific behavior.
```

Q: We did not pass payment type to pay(). How is it chosen?
The factory already returned UpiPayment. The reference type is Payment, but the actual object determines which overridden method runs through runtime polymorphism.

Q: Can both patterns work together?
Yes. Factory provides the processor; Strategy organizes the alternative processing behaviors. Neither requires the other.

Q: Does Strategy choose the pay method?
More precisely, a caller, configuration or selector chooses the strategy object. Strategy defines the alternative behaviors; Java dispatch executes the selected object's method.

Q: Can we use Strategy without Factory?
Yes: new CheckoutService(new UpiPayment()).checkout(1000). The processor was supplied directly.

A real order example

Customer selects UPI for a 1,000-rupee order. The application passes UPI to the factory. The factory provides UpiPayment. Checkout calls pay(1000). The UPI processor performs provider-specific processing. The sample printing code only demonstrates dispatch; it is not a real payment integration.

One-line revision

Factory provides the object. Strategy provides interchangeable behavior.

6. Builder - What, Why and Advantages

1. What is it?

A helper that collects named values and creates the final object with build().

2. What was the problem before?

A long constructor such as Product("Laptop", "Dell", 50000, true, false) is hard to read. Same-typed arguments can be swapped accidentally.

3. Why did we introduce it?

Use methods such as name(), brand() and inStock() so each supplied value has a visible meaning.

4. What are the advantages?

Readable construction; less argument-position confusion; convenient optional values; avoids many constructor combinations.

5. Where exactly does it fit?

Order Service builds OrderResponse with orderId, amount, status and customer details before returning it to React.

6. When should we be careful?

More code or generated machinery. Required-field validation and immutability are not automatic. Prefer constructors for a few obvious values.

Continue to the next page for the Java example, detailed usage and interview questions.

6. Builder - Example and Interview Questions

What and why

Builder collects values through named methods, then constructs the final object with build(). It improves readability when a class has many fields or many optional combinations.

```
// Constructor: meanings depend on argument position.
new Product("Laptop", "Dell", 50000, true, false);

// Builder: meanings appear next to the values.
Product product = Product.builder()
    .name("Laptop")
    .brand("Dell")
    .price(50000)
    .inStock(true)
    .freeDelivery(false)
    .build();
```

With Lombok, @Builder can generate this API. A plain Java implementation is also possible; Builder is not dependent on Lombok.

```
@Getter
@Builder
public class OrderResponse {
    private Long orderId;
    private String customerName;
    private int amount;
    private String status;
}
// Example inside Order Service:
OrderResponse response = OrderResponse.builder()
    .orderId(101L)
    .customerName("Lakshmi")
    .amount(1000)
    .status("CREATED")
    .build();
```

Exact project usage

Preparing OrderResponse DTOs for React or constructing event payloads with several fields. builder() creates a helper; named calls populate it; build() creates the response. Requires Lombok imports and annotation processing in this example.

Q: Constructor versus Builder?
Constructor parameters follow a fixed positional signature. Builder methods name fields and can usually be called in any order. Builder commonly invokes a constructor internally.

Q: Why is swapping constructor arguments risky?
new Product("Dell", "Laptop", ...) can compile if both parameters are String, while assigning the wrong meanings. Named methods make the intended assignment visible.

Q: Builder versus setters?
Setters modify an already created object. A builder collects values before producing the final object. Both can be readable; neither is always best.

Builder - Advantages and Follow-ups

Q: What is the main advantage?
Readable object creation, less confusion over argument positions, and convenient optional fields. Benefits grow as the number of fields or combinations increases.

Q: Can I omit fields?
Usually yes, but omitted fields take defaults such as null, zero or false unless configured otherwise. Required-field validation must be added deliberately.

Q: Does Builder make the object immutable?
No. Immutability requires a suitable class design, including controlled mutation and defensive copies where needed.

Q: Does Builder replace constructors?
No. It provides a construction API and commonly calls a constructor internally.

Q: When is a constructor better?
For a small number of obvious required values, a constructor is often simpler and avoids extra builder machinery.

Q: Factory versus Builder?
Factory provides an appropriate object or type. Builder helps assemble an object with specified values step by step.

Interview answer

"Builder constructs objects step by step using named methods. It is useful for an OrderResponse with several required and optional fields because the code is easier to read than a long constructor call."

Plain Java miniature

```
class OrderResponse {
    private final long id;
    private final String status;
    private OrderResponse(Builder b) {
        id = b.id;
        status = b.status;
    }
    static Builder builder() { return new Builder(); }
    static class Builder {
        private long id;
        private String status;
        Builder orderId(long id) {
            this.id = id; return this;
        }
        Builder status(String status) {
            this.status = status; return this;
        }
        OrderResponse build() {
            return new OrderResponse(this);
        }
    }
}
```

7. Adapter - What, Why and Advantages

1. What is it?

A wrapper that converts an existing interface into the one your application expects.

2. What was the problem before?

Business code may need pay(rupees), while a provider SDK exposes makePayment(paise). Direct calls spread provider-specific conversions across the application.

3. Why did we introduce it?

Put translation in one adapter implementing your PaymentGateway interface.

4. What are the advantages?

Central conversion rules; reduced provider coupling; original SDK stays unchanged; adapter can be tested independently.

5. Where exactly does it fit?

PaymentAdapter converts 100 rupees to 10,000 paise and invokes the external client.

6. When should we be careful?

Conversion and error mapping must be correct. The adapter needs maintenance when the provider API changes.

Continue to the next page for the Java example, detailed usage and interview questions.

7. Adapter - Example and Interview Questions

What and why

Adapter makes an existing interface work with the interface your application expects. Like a plug adapter, it bridges a mismatch without changing the original component.

```
interface PaymentGateway {
    void pay(int rupees);
}
class ExternalPaymentClient {
    void makePayment(int paise) {
        System.out.println("Provider received " + paise);
    }
}
class PaymentAdapter implements PaymentGateway {
    private final ExternalPaymentClient client;
    PaymentAdapter(ExternalPaymentClient client) {
        this.client = client;
    }
    public void pay(int rupees) {
        client.makePayment(Math.multiplyExact(rupees, 100));
    }
}
// Inside main:
PaymentGateway gateway =
    new PaymentAdapter(new ExternalPaymentClient());
gateway.pay(100); // Provider received 10000
```

Exact project usage

Between Payment Service business logic and an external provider SDK. The adapter converts your method calls, request fields, amount units and provider responses. This provider API is illustrative, not a claim about a particular SDK.

Benefits: provider-specific conversion stays in one place, business logic uses a stable interface, and the external SDK does not need modification. Real integrations must validate amounts and correctly map errors.

Q: Is every external API call an Adapter?
No. Adapter specifically bridges an interface mismatch. A direct call using the provider API is not automatically this pattern.

Interview answer

"Adapter translates an existing interface into the interface our application expects. A payment adapter can convert our request into the provider SDK's method calls and data format."

8. Decorator - What, Why and Advantages

1. What is it?

A wrapper that keeps the same interface while adding behavior.

2. What was the problem before?

Adding logging directly to every payment processor duplicates code. Creating a new subclass for each combination of logging and timing grows unwieldy.

3. Why did we introduce it?

Wrap the processor with a logging or timing decorator and delegate the actual payment.

4. What are the advantages?

Original processor stays focused; optional behaviors can be combined; avoids a subclass for every combination.

5. Where exactly does it fit?

LoggingPaymentDecorator wraps CardPayment and logs around pay(amount).

6. When should we be careful?

Wrappers add indirection. Ordering matters, and the wrapper must preserve the operation contract and exceptions.

Continue to the next page for the Java example, detailed usage and interview questions.

8. Decorator - Example and Interview Questions

What and why

Decorator wraps an object through the same interface and adds behavior. It lets you add logging or timing while keeping the original payment class unchanged.

```
interface Payment {
    void pay(int amount);
}
class CardPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Card payment: " + amount);
    }
}
class LoggingPaymentDecorator implements Payment {
    private final Payment payment;
    LoggingPaymentDecorator(Payment payment) {
        this.payment = payment;
    }
    public void pay(int amount) {
        System.out.println("Payment started");
        payment.pay(amount);
        System.out.println("Payment completed");
    }
}
// Inside main:
Payment payment = new LoggingPaymentDecorator(
    new CardPayment());
payment.pay(1000);
```

Output: Payment started; Card payment: 1000; Payment completed. If the wrapped call throws, the final line is not reached; use suitable exception/finally handling for always-run cleanup or timing.

Exact project usage

A possible wrapper inside Payment Service for logging or measurement. A standard Java example is BufferedInputStream wrapping FileInputStream to add buffering. Multiple decorators can be composed.

Q: Adapter versus Decorator?
Adapter changes an interface to make it compatible. Decorator retains the interface and adds behavior.

Q: Proxy versus Decorator?
Both may wrap and delegate. Proxy primarily represents or controls access to an object; Decorator primarily composes additional responsibilities. Intent distinguishes them.

Interview answer

"Decorator adds behavior by wrapping an object through the same interface. A logging payment decorator can add logs around payment processing without changing the underlying processor."

9. Publish-Subscribe - What, Why and Advantages

1. What is it?

Publish an event so independent subscribers can react.

2. What was the problem before?

If Payment directly calls both Order and Notification, it must know both endpoints and handle their availability in the request workflow.

3. Why did we introduce it?

Publish payment-success and let the interested services consume it independently.

4. What are the advantages?

Reduced direct coupling; subscribers scale and progress independently; more subscribers can be added without adding publisher calls.

5. Where exactly does it fit?

Order and Notification use different Kafka consumer groups to independently consume payment-success.

6. When should we be careful?

Events may be redelivered; handle failures and idempotency. Separate groups do not guarantee processing order between services.

Continue to the next page for the Java example, detailed usage and interview questions.

9. Publish-Subscribe - Example and Interview Questions

What and why

A publisher emits an event and independent subscribers react. Payment Service can publish payment-success without directly calling Order and Notification services.

```
// Payment Service publisher (String example):
kafkaTemplate.send("payment-success", "ORDER-101");

// Order Service:
@KafkaListener(topics = "payment-success",
               groupId = "order-service")
public void confirmOrder(String orderId) {
    // Update the order in the database.
}

// Notification Service:
@KafkaListener(topics = "payment-success",
               groupId = "notification-service")
public void notifyCustomer(String orderId) {
    // Send customer notification.
}
```

Exact project usage

Payment publishes payment-success. Order consumes it to confirm the order. Notification consumes it to notify the customer. Production events generally include an event ID, order ID and other required fields.

Q: Why different consumer groups?
Different groups independently consume the topic. Inside one group, partitions are assigned among its consumers, so the group shares the work rather than delivering every event to each member.

Q: What if a subscriber is down?
It can resume from its committed position if the needed records are still retained and configuration permits. Retention, offset management and error handling matter.

Q: Does Kafka guarantee business processing happens once?
Not automatically. Failures and retries can lead to redelivery; design handlers to be idempotent. Publishing is also asynchronous, so handle send failures.

Q: Do both subscribers run in order?
No. Separate consumer groups progress independently. A notification can be processed before the order update unless the workflow explicitly coordinates that dependency.

Interview answer

"I use Kafka publish-subscribe. Payment Service publishes payment-success, and Order and Notification consume it with separate groups. This reduces direct dependencies and allows independent processing."

10. Saga Choreography - What, Why and Advantages

1. What is it?

A workflow of local transactions with compensating actions, coordinated through events.

2. What was the problem before?

Stock may be reserved before payment fails. Rolling back the Payment database cannot reverse an already committed Product database change.

3. Why did we introduce it?

Define business compensation: release stock and cancel the order when payment fails.

4. What are the advantages?

Explicit failure recovery across services; avoids relying on one cross-service ACID transaction; services own local transactions.

5. Where exactly does it fit?

In a stock-reservation flow, payment-failed triggers stock release and order cancellation. Implement this only if stock was previously reserved.

6. When should we be careful?

Eventually consistent, more complex recovery. Compensations can fail and need retry, idempotency and sometimes manual intervention.

Continue to the next page for the Java example, detailed usage and interview questions.

10. Saga Choreography - Example and Interview Questions

What and why

Saga coordinates a business operation across services through local transactions. When a later step fails, compensating actions reverse earlier business effects where possible. A local @Transactional method cannot automatically undo commits in other services.

Illustrative success flow with stock reservation

1. Order creates a CREATED order and publishes order-created.
2. Product reserves stock and publishes stock-reserved.
3. Payment consumes stock-reserved and processes payment.
4. Payment publishes payment-success.
5. Order consumes it and marks the order CONFIRMED.

Failure and compensation

If payment fails, Payment publishes payment-failed. Product releases the reserved stock, and Order marks the order CANCELLED. Each compensation is another local business operation, not a rollback of the earlier committed database transaction.

Exact project usage

Your simpler discussed flow is order-created -> Payment -> payment-success -> Order CONFIRMED, with Notification independently consuming payment-success. The stock-reservation flow above is an extended example. Explain stock restoration only when your code actually reserves or reduces stock and implements its release.

Q: Choreography versus Orchestration?
Choreography: services react to events without a central coordinator. Orchestration: a coordinator sends commands and tracks workflow progress.

Q: Is Kafka the same as Saga?
No. Kafka transports events. Saga defines the business sequence and compensation rules. Kafka alone does not implement those rules.

Q: Is compensation a database rollback?
No. It is a new committed operation such as releasing a reservation or issuing a refund. Some effects cannot be perfectly reversed.

Q: What if compensation fails?
Use reliable retry and idempotency, track the failure, and provide recovery or manual intervention. Compensation is not guaranteed merely because an event was published.

Q: Does Saga provide immediate consistency?
Usually the workflow is eventually consistent. Other services may temporarily show intermediate states.

Interview answer

"Saga coordinates local transactions across services. If a step fails, compensation reverses earlier business effects. With choreography, services coordinate through events. For example, failed payment can trigger stock release and order cancellation."

Final Interview Revision

Which patterns have you used?

"I use Repository through Spring Data JPA to separate persistence from business logic. Spring provides singleton-scoped service beans and proxies for transaction management. Feign also provides client proxies. For asynchronous communication, I use Kafka publish-subscribe. Our order/payment workflow follows choreography, with compensation where implemented."

Add Strategy, Factory, Builder, Adapter or Decorator only when you can explain their actual location in your code. Otherwise explain them as designs you understand or could introduce.

One-line memory aid

Singleton: One shared instance.
Repository: Separate persistence access.
Proxy: Intermediary around the real object.
Strategy: Interchangeable behavior.
Simple Factory: Central object creation/selection.
Builder: Named values, then build.
Adapter: Make incompatible interfaces work together.
Decorator: Wrap and add behavior.
Publish-Subscribe: Publish events for independent subscribers.
Saga: Local transactions plus compensation.

Are these enough? Which other patterns matter?

For an initial interview answer, three or four patterns with concrete explanations are stronger than a long unsupported list. Strategy, Factory and Builder are important Java examples; understand Proxy and Singleton in Spring, and Adapter for integrations. This planned set is complete, but it is not an exhaustive list of all design patterns.

How to explain any pattern

State the problem, explain the pattern, identify the class or service where it applies, and describe its benefit. Be ready to explain one limitation. Keep the initial answer short; expand with code when asked.